

How Computers Execute Programs

Lecture 2

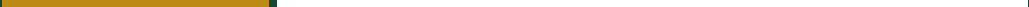
CS 1001

Kirk Duran

Today

- ▶ What a computer is and why computers are useful.
- ▶ How computing developed from counting tools to modern electronic systems.
- ▶ How a computer stores programs, executes instructions, and represents information.
- ▶ How machine language, assembly, and higher-level programming languages connect.
- ▶ What computer science studies and where computers appear in society.
- ▶ Why computation has a physical cost in hardware, energy, and infrastructure.

Part 1: What Is a Computer?



What Are Computers?

Definition

A computer is a programmable electronic device that processes data to perform tasks.

Image placeholder: `computer-components.jpg`

- ▶ **Input:** information enters the system.
- ▶ **Processing:** the CPU follows instructions and transforms information.
- ▶ **Memory/storage:** information is kept for immediate or later use.
- ▶ **Output:** the system produces a result or physical action.

Why Do We Use Computers?

- ▶ **Speed:** computers can perform enormous numbers of simple operations quickly.
- ▶ **Automation:** repetitive procedures can run with little human intervention.
- ▶ **Accuracy and repeatability:** the same instructions can be followed consistently.
- ▶ **Scale:** computers let us analyze, store, and communicate more information than a person could manage manually.

Example

Applications include scientific simulation, genome analysis, CAD, robotics, gaming, communication, and business systems.

Image placeholder: clerical-computation.jpg

Key idea

Computers amplify human procedures; they do not remove the need to decide which procedures make sense.

Part 2: A Brief History of Computing

From Counting Tools to Algorithms

Antiquity

The abacus supported arithmetic by giving people a physical representation of quantities and operations.

Image placeholder: abacus.jpg

9th Century

Persian mathematician Muhammad ibn Musa al-Khwarizmi wrote systematic procedures for calculation. His name is the historical root of the word *algorithm*.

Image placeholder: al-khwarizmi.jpg

Key idea

Computing has always involved both **representations** of information and **procedures** for manipulating it.

Mechanical Calculation and Programmable Machines

17th Century

Blaise Pascal and Gottfried Wilhelm Leibniz developed mechanical calculators for arithmetic operations.

Image placeholder: mechanical-calculator.jpg

1830s

Charles Babbage designed the Analytical Engine as a general-purpose mechanical computer. Ada Lovelace described an algorithm intended for the machine.

Image placeholder: babbage-engine.jpg

1936: Turing's Abstract Computer

- ▶ Alan Turing introduced a mathematical model now called a **Turing machine**.
- ▶ The model showed how a small set of simple operations could express arbitrary algorithms.
- ▶ The important idea is not the physical machine; it is a model of what computation itself can do.

Image placeholder: `turing-machine.jpg`

Definition

Informally, a system is **Turing complete** when it can simulate any computation that a Turing machine can perform, given enough time and memory.

Example

This is about computational expressiveness, not whether a system is convenient, fast, or intended to be used as a computer.

1940s–1950s: Electronic Computers and Languages

Image placeholder: eniac.jpg

Image placeholder: punch-card.png

1940s

Electronic machines such as ENIAC could perform complex calculations much faster than manual calculation.

1950s

Languages such as FORTRAN and Lisp changed how humans expressed programs, moving programmers farther from raw machine instructions.

1970s–1990s: Computers Become Personal and Connected

Personal Computing

Microprocessors made smaller, less expensive computers practical. Systems such as the Apple I helped move computing toward individual users.

Image placeholder:

`early-personal-computer.jpg`

The Internet and Web

Networked computers transformed communication and data exchange. Tim Berners-Lee's World Wide Web made linked information much easier to publish and access.

Image placeholder: `internet-network-map.jpg`

21st Century: Mobile, Cloud, and AI

- ▶ Computing moved from one machine on one desk to networks of devices and data centers.
- ▶ Cloud platforms let organizations rent computation and storage at large scale.
- ▶ Smartphones put general-purpose computers, cameras, sensors, and networks in billions of pockets.
- ▶ AI systems increasingly use large-scale computing for perception, prediction, language, and control.

Image placeholder: ai-era.jpg

Key idea

Modern computing is often a **system of computers**, not a single computer.

Part 3: How Computers Execute and Represent Information



The Stored-Program Computer

Key idea

A modern general-purpose computer stores both **data** and **program instructions** in memory.

- ▶ **CPU:** executes instructions.
- ▶ **Memory:** holds information being actively used.
- ▶ **Input/output:** connects the machine to users, devices, files, and networks.
- ▶ **Storage:** keeps information when programs are not actively using it.

Image placeholder:

`stored-program-architecture.jpg`

Inside the CPU: Control, Arithmetic, and Registers

- ▶ The **control unit** coordinates instruction execution.
- ▶ The **arithmetic/logic unit (ALU)** performs arithmetic and logical operations.
- ▶ **Registers** are tiny, very fast storage locations inside the processor.
- ▶ Main memory is much larger than the registers but typically slower to access.

Image placeholder: computer-components.jpg

Example

A useful mental model: the CPU is the worker, registers are the worker's hands, memory is the nearby workbench, and long-term storage is the filing cabinet.

The Instruction Cycle: Fetch, Decode, Execute

1. **Fetch:** retrieve the next instruction from memory.
2. **Decode:** determine what the instruction means.
3. **Execute:** perform the requested operation.
4. Store intermediate or final results where they are needed.
5. Repeat.

Image placeholder: instruction-cycle.jpg

In class

Imagine the instruction is “add the values in A and B and save the result in C.” What must be fetched, processed, and stored?

Do Computers “Think”?

- ▶ Traditionally, a CPU does not decide what its instructions *should* mean. It executes operations defined by its architecture and program.
- ▶ This precision is useful: the same operation can be repeated consistently.
- ▶ It is also dangerous: flawed instructions can be followed just as faithfully as correct ones.

Key idea

Computers are physical systems that implement rules. Human judgment enters through the problems, programs, data, and goals we give them.

Image placeholder:

lukyanov-water-computer.jpg

Lukyanov's hydraulic integrator reminds us that computation is not inherently electronic.

Computer Memory: Locations and Addresses

- ▶ Think of memory as a large collection of storage locations.
- ▶ Each location has an **address** that identifies where data can be found.
- ▶ Addresses are numbers; programmers often see them written in hexadecimal.
- ▶ Programs can store a value at an address and later retrieve it.

Image placeholder: `memory-addresses.jpg`

Example

The address identifies the location. The value is the information stored at that location.

Bits and Bytes Are Physical

Definition

A **bit** is one binary digit: 0 or 1. A **byte** is typically 8 bits.

- ▶ Modern computers represent bits using physical electronic states, commonly implemented with transistors and related circuitry.
- ▶ The exact technology can change; the logical abstraction of 0 and 1 remains useful.
- ▶ Bytes can encode numbers, characters, portions of images, instructions, and many other kinds of information.

Image placeholder: `memory-cells-bits.jpg`

Memory Scale and Memory Technology

Scale

- ▶ 1 byte = 8 bits
- ▶ KB: thousands of bytes
- ▶ MB: millions of bytes
- ▶ GB: billions of bytes
- ▶ TB: trillions of bytes

Key idea

Large datasets and models can require enormous memory even though each bit stores almost nothing.

Image placeholder: `magnetic-core-memory.jpg`

Image placeholder: `ram-module.jpg`

Technology

Earlier magnetic-core systems were physically large and expensive. Semiconductor memory stores vastly more information in far less space and at much lower cost per byte.

Image placeholder: `memory-warehouse.jpg`

Why Binary?

- ▶ Binary is a base-2 numeral system containing only 0 and 1.
- ▶ Digital hardware can reliably distinguish between two ranges of physical states.
- ▶ Transistors act like controlled electronic switches and are combined to build logic gates, memory, and processors.

Image placeholder: transistor-switch.jpg

Example

Binary is not “the language of electricity.” It is a convenient logical representation implemented by physical devices.

Reading Binary Numbers

Place values

From right to left, binary places represent 1, 2, 4, 8, 16, 32, and so on.

Example

$$0101 = 0(8) + 1(4) + 0(2) + 1(1) = 5.$$

In class

What decimal number is 1010? What binary representation would you use for decimal 7?

Image placeholder:
`machine-code-cpu.jpg`

Binary Addition Works Like Familiar Addition

- ▶ Add one column at a time.
- ▶ $0 + 0 = 0$
- ▶ $0 + 1 = 1$
- ▶ $1 + 1 = 10$: write 0 and carry 1.
- ▶ Carries propagate just as they do in base 10.

Image placeholder: `binary-addition.jpg`

Example

$0101\ (5) + 0011\ (3) = 1000\ (8).$

Part 4: From Machine Language to Python

Machine Language: Instructions the Processor Executes

- ▶ A processor understands a particular set of machine instructions.
- ▶ Machine code represents those instructions as bit patterns.
- ▶ Instructions can move data, perform arithmetic, compare values, jump to other instructions, and interact with hardware.
- ▶ Raw machine code is difficult for humans to write and inspect directly.

Image placeholder: machine-code-cpu.jpg

Assembly Language Gives Machine Instructions Names

Definition

Assembly language is a low-level programming language whose instructions closely correspond to a processor's machine instructions.

- ▶ Mnemonics such as MOV and ADD are easier for humans than raw binary.
- ▶ An **assembler** translates assembly instructions into machine code.
- ▶ Different processor architectures have different instruction sets.

Image placeholder:

assembly-to-machine-code.jpg

Higher-Level Languages Automate the Translation Work

- ▶ Early programmers sometimes translated symbolic instructions into machine code manually.
- ▶ Assemblers automated one layer of translation.
- ▶ Compilers and interpreters let programmers express algorithms using increasingly human-friendly abstractions.
- ▶ Languages such as FORTRAN, C, Java, and Python let us focus more on the problem and less on processor details.

Image placeholder: `compiler-automation.jpg`

Key idea

Programming languages are layers of abstraction over the operations the machine ultimately performs.

Why Learn to Code When Tools Can Generate Code?

- ▶ Automation repeatedly removes some low-level work and creates new higher-level work.
- ▶ Today's AI tools can write boilerplate, summarize information, and help debug programs.
- ▶ Someone still has to define the problem, judge whether the result is correct, test edge cases, and understand consequences.

Image placeholder: `high-level-code.jpg`

Key idea

The valuable skill is not merely typing syntax. It is being able to describe computation precisely enough to inspect, test, and control it.

Part 5: Computer Science and Computer Systems

What Is Computer Science?

Definition

Computer science studies computation: algorithms, data, programming languages, systems, and the limits and applications of computation.

- ▶ **Theory:** algorithms and computational complexity.
- ▶ **Systems:** operating systems, databases, networks, architecture.
- ▶ **Software:** programming languages, software engineering, design.
- ▶ **Applied computing:** AI, machine learning, graphics, robotics, security, data science, HCI, and more.

Image placeholder: `cs-fields-taxonomy.jpg`

Computer Science Changes With Society

Image placeholder: `cs-degree-history.jpg`

Participation changes

Enrollment and degree patterns have risen and fallen with technology cycles, labor markets, culture, and access to computing education.

Image placeholder: `tech-jobs-history.jpg`

Jobs change too

Computing occupations evolve as technology changes. Historical charts are snapshots, not guarantees about future demand.

Who Uses Computers?

- ▶ **General users:** communication, entertainment, school, work, finance, navigation.
- ▶ **Scientists:** simulation and data analysis.
- ▶ **Engineers:** embedded systems, CAD, control, testing.
- ▶ **Businesses:** databases, logistics, CRM/ERP, e-commerce.
- ▶ **Accessibility:** screen readers, communication aids, prosthetic control.

Key idea

Because computing reaches nearly everyone, privacy, security, accessibility, bias, and responsible design are technical concerns too.

Image placeholder: `smartphone-users-2016-2022.jpg`

Historical smartphone-adoption snapshot from the original lecture.

Are All Computers the Same?

General-purpose

PCs and smartphones are designed to run many different programs.

Specialized

GPUs, embedded controllers, and supercomputers may be optimized for particular workloads or environments.

Distributed

Clusters and cloud systems divide work across many networked machines for scale and fault tolerance.

Image placeholder: `personal-computer-timeline.jpg`

Quantum Computers and Emerging Directions

- ▶ **Quantum computing:** qubits and quantum effects enable different algorithms; potential speedups apply to particular problem structures rather than every task.
- ▶ **Specialized hardware:** GPUs and other accelerators trade generality for efficiency on important workloads.
- ▶ **New architectures:** research explores neuromorphic, photonic, and other post-Moore approaches.
- ▶ **Software and systems:** AI frameworks, cloud systems, distributed computing, and edge computing spread computation across many kinds of devices.
- ▶ **Efficiency:** power consumption and cooling increasingly constrain what systems can do.

Image placeholder: quantum-computer.jpg

Example

Shor's algorithm is a famous example where a sufficiently capable quantum computer would change the complexity of integer factoring.

Part 6: Computation Has a Physical Cost

Activity: Three Ways to Get a Times Table

In class

Suppose you want the complete multiplication table from 1 through 12. Which approach do you expect to use the most electricity?

1. Write a small nested loop and compute all 144 products locally.
2. Search the web and retrieve a page containing the table.
3. Ask a large AI system to generate the program and return the results.

Example

Ignore your own time for the moment. Think about the computation, networking, storage, and infrastructure involved behind each option.

The Energy Comparison Is About Scale, Not Exact Numbers

- ▶ A few arithmetic operations on a local processor require very little energy.
- ▶ A network query activates networking equipment, servers, storage, cooling, and other infrastructure.
- ▶ A large AI inference may perform far more computation than the tiny arithmetic task itself.
- ▶ Exact energy use depends on hardware, software, location, batching, model size, utilization, and many other details.

Image placeholder: cloud-data-center.jpg

Key idea

Use the simplest tool that solves the problem well. Abstraction is valuable, but abstraction is not physically free.

Computing Depends on Physical Infrastructure

- ▶ Data centers require electricity, networking, cooling, land, and hardware.
- ▶ Growing computing demand can affect decisions about power generation and transmission.
- ▶ This is another reason efficiency matters: faster or smaller algorithms can reduce real resource use at scale.

Image placeholder:

data-center-energy-headline.jpg

Key idea

Software looks abstract, but every program ultimately runs on physical machines somewhere.

Energy Production Is Also a Computing Problem

- ▶ Electrical demand changes throughout the day.
- ▶ Renewable generation also changes with conditions such as sunlight and wind.
- ▶ Grid operators use sensing, forecasting, optimization, and control software to balance production and demand.
- ▶ The “duck curve” illustrates how net demand can fall during solar-heavy hours and rise rapidly later in the day.

Image placeholder: duck-curve.png

Where Python Fits

What you write

```
print("Hello, world!")  
x = 5 + 3
```

What the computer eventually needs

Operations represented in forms the processor and operating system can execute.

Key idea

Python lets us work at a high level while software underneath handles many details of memory, machine instructions, files, and hardware.

Before next time

- ▶ Make sure you can open Python or Google Colab.
- ▶ Create, save, and find a notebook or source file.
- ▶ Remember: Python program → translated operations → machine execution.

In class

Bring one question about something inside a computer that still feels mysterious.

Next Time

Python Introduction